



CSharp Pro

(Let's go, but not by "go", by C#)

Parham Darvishi

Linkedin: <https://www.linkedin.com/in/parham-darvishi-26b6b8161/>

Github: <https://github.com/parhamdarvishi>

Phone Number: +989129564205

Email: darvishiparham14@gmail.com

Telegram Channel: [codemasters](#)Plus

Windows applications

ایجاد واسط گرافیکی

- آشنایی با فرم
- آشنایی با انواع کنترلرها
- کنترلرهای ساده
- کنترلرهای گروه بندی
- کنترلرها گسترش دهنده
- منوها و نوار ابزارها

آشنایی با اجزای فرم

- کلاس فرم
- کلاس Designer فرم
- آشنایی با Resource

Windows Application

- Load
- Activated
- Disactivated
- FromClosing
- KeDown
- KeyUp
- KeyPress
- MouseEvent

نمایش فرم

- متد Show
- متد ShowDialog
- رویدادهای مهم:

Value Type & Reference Type

Check Dotnet Fundamental Presentation

Programming Paradigms (الگوها)

These paradigms provide different approaches to **structuring** and **designing programs**, each with its own strengths and weaknesses depending on the problem domain and requirements.

- **Procedural Programming:** In this paradigm, programs are organized around **functions or procedures** that perform **operations on data**. It's based on the concept of procedure calls, where functions are defined to perform specific tasks. **C and Pascal** are examples of languages that support procedural programming.
- **Functional Programming:** This paradigm focuses on treating computation as the evaluation of mathematical functions and avoids changing state and mutable data. Functions are first-class citizens, meaning they can be **passed as arguments and returned as values**. Languages like Haskell, Lisp, and Erlang are known for supporting functional programming.
- **Imperative Programming:** In this paradigm, the focus is on **specifying the sequence of statements** that change a program's state. It **encompasses both procedural and object-oriented programming**. Languages like C and Java (when used in an imperative style) fall into this category.
- **Declarative Programming:** This paradigm emphasizes expressing the desired behavior **without explicitly specifying the steps** to achieve it. **SQL**, for example, is a declarative language used to query databases. Markup languages like **HTML** are also declarative.
- **Logic Programming:** Logic programming involves defining a set of facts and rules, and then posing queries to a system to infer new facts or relationships. Prolog is a popular language for logic programming.

- **Event-Driven Programming:** In this paradigm, programs are designed to respond to events, such as user input or system events. It's common in graphical **user interfaces and real-time systems**. Languages like **JavaScript** (in web development) heavily use event-driven programming.
- **Concurrent Programming:** This paradigm involves handling multiple tasks or processes concurrently, allowing programs to efficiently utilize multi-core processors and handle tasks in parallel. Languages like **Go** and Erlang are designed to make concurrent programming easier.
- **Parallel Programming:** Similar to concurrent programming, parallel programming focuses on dividing a problem into smaller tasks that can be executed simultaneously on multiple processors or cores. High-performance computing often involves parallel programming.
- **Aspect-Oriented Programming (AOP):** AOP complements OOP by allowing developers to modularize cross-cutting concerns, like logging or security, into separate units called "aspects." This helps in achieving better separation of concerns.
- **Dataflow Programming:** In this paradigm, computation is modeled as a directed graph, where nodes represent processing units and edges represent data dependencies. The flow of data through the graph dictates the program's execution.
- **Reactive Programming:** Reactive programming focuses on managing and reacting to asynchronous data streams and events. It's particularly useful in scenarios involving real-time data, such as user interfaces and streaming data applications.

Software Design Principle:

SOLID Principles: SOLID stands for Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion. These principles aim to make software designs more robust, scalable, and maintainable by focusing on separation of concerns, modularity, and loosely coupled components.

KISS Principle: KISS stands for **Keep It Simple, Stupid**. It emphasizes the importance of **simplicity in software design**, which leads to easier maintenance, debugging, and testing.

DRY Principle: DRY stands for **Don't Repeat Yourself**. It encourages software developers to avoid duplication of code, which can lead to inconsistencies and maintenance issues.

YAGNI Principle: YAGNI stands for **You Ain't Gonna Need It**. It advises developers to **avoid adding unnecessary features or functionality** to software designs, as they can complicate the codebase and increase maintenance costs.

Object Oriented Programming

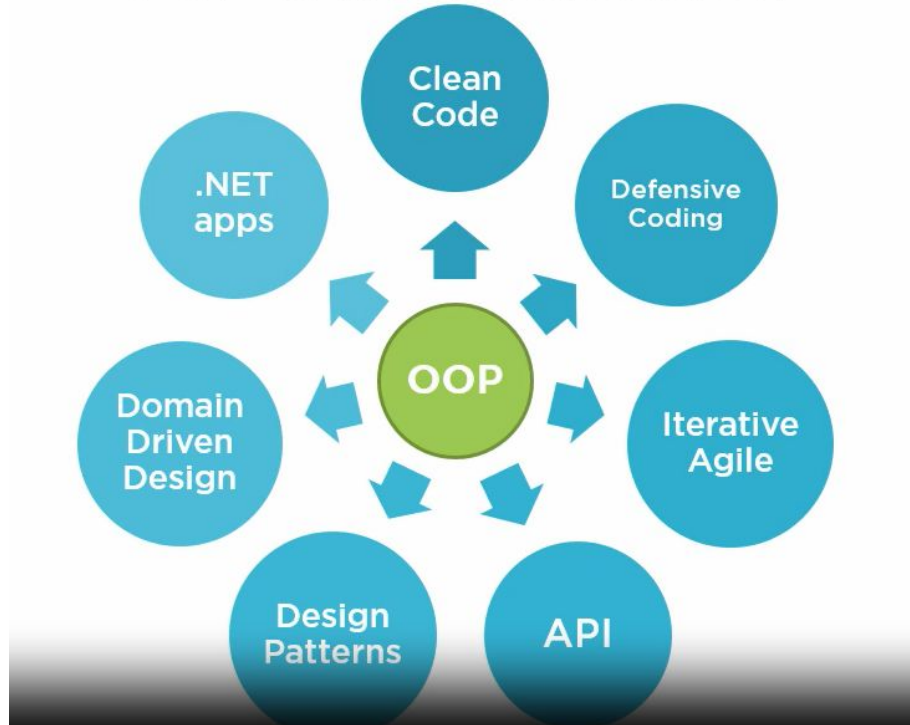
OOP is one of the most popular and widely used **paradigms** in software development.

Object-oriented programming has several advantages over procedural programming:

- OOP is faster and easier to execute
- OOP provides a clear structure for the programs
- OOP helps to keep the C# code DRY "Don't Repeat Yourself", and makes the code easier to maintain, modify and debug
- OOP makes it possible to create full reusable applications with less code and shorter development time

OOP

OOP Is the Foundation



Object-Oriented Programming (OOP)

An approach to designing and building applications that are:

- Flexible
- Natural
- Well-crafted
- Testable

by focusing on objects that interact cleanly with one another

Identifying
classes

Separating
responsibilities

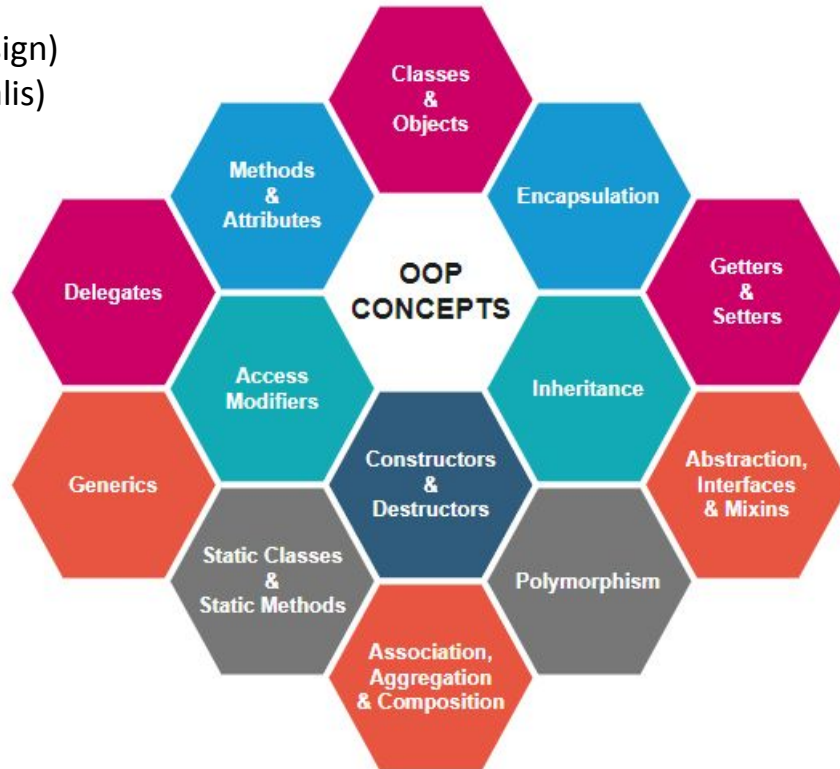
Establishing
relationships

Leveraging
reuse

OOP Concepts

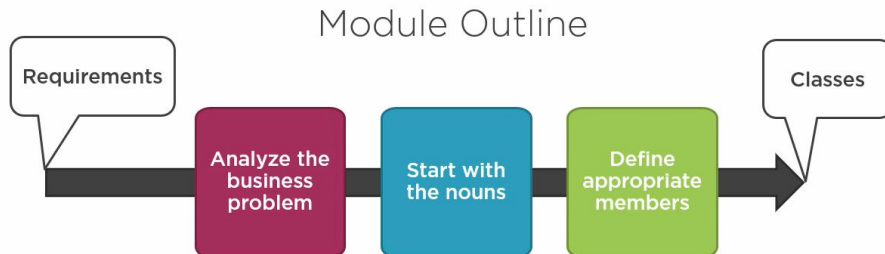
OOD (Object Oriented Design)

OOA (Object Oriented Analysis)



How to develop OOP applications?

- برنامه ای که قصد توسعه آن را داریم، تصور میکنیم.
- در مرحله اول باید موجودیت ها را شناسایی کنیم. (Entity)
 - سوال: چگونه موجودیت را تشخیص دهیم؟
 - روش اول: شناسایی مفاهیم دنیای واقعی که معمولا از **اسامی** که دامین به ما میگوید از آن مطلع میشویم. (شنیدن داستان پروژه)
 - روش دوم: brainstorming
- میخوایم برنامه ای برای رستوران بنویسیم، یا برای آموزشگاه یا برای داروخانه یا برای سیستم ورود و خروج
- حالا من موجودیت هام رو درآوردم وقت این است مه خصوصیت های (Property) هر موجودیت رو در بیارم.
 - هر سیستم عملیاتی خصوصیت خاص خود را دارد.
 - در بین موجودیت ها دنبال به چیزی باشید که خصوصیتتون را منحصر به فرد کند مثل شماره ملی برای افراد.
- رابطه بین موجودیتها
- گاهی اوقات یک روند را خودش یک موجودیت میگیریم. مثل موجودیت ترم در سیستم آموزشی (این نوع موجودیتها به صورت تحلیل در مبارد)
- هر موجودیت یک سری فرآیند یا رفتار (Method) هم دارد که **فعل** هستند. (کارهایی که آن موجود می تواند انجام دهد).
- تعریف class یک موجودیت (اصلی ترین کلمه شی گرای هستند).



Define Appropriate Members

Customer

- Name
- Email address
- Home address
- Work address
- Validate()
- Retrieve()
- Save()

Product

- Product name
- Description
- Current price
- Validate()
- Retrieve()
- Save()

Order

- Customer
- Order date
- Shipping address
- Order items
- Validate()
- Retrieve()
- Save()

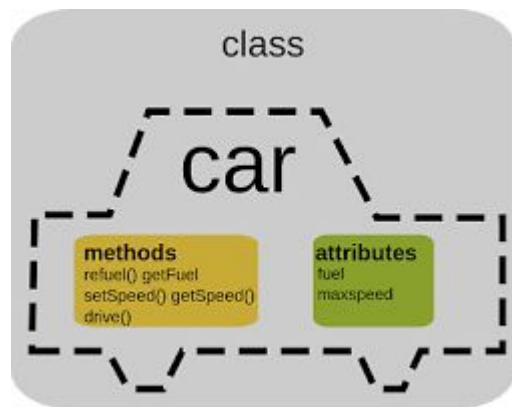
Order Item

- Product
- Quantity
- Validate()
- Retrieve()
- Save()

Entity

موجودیت هر آن چیزی است که در یک سیستم عملیاتی به صورت مستقل وجود دارد و با ارزش است.

Class

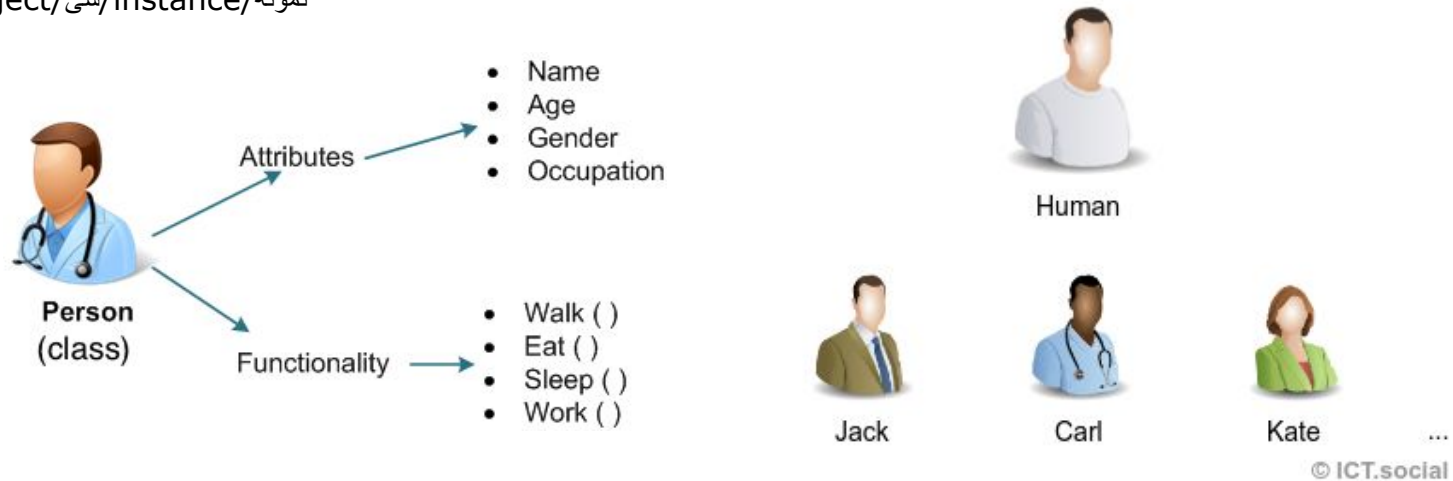


Class & Object

Class is a template for objects, and an object is an instance of a class.

Object/شیء/instance/نمونه

Go



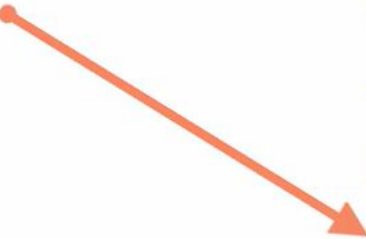
Object

!=

Class

```
var customer = new Customer();  
customer.FirstName = "Frodo";  
customer.Validate();
```

```
public class Customer  
{  
    public int CustomerId { get; set; }  
    public string EmailAddress { get; set; }  
    public string FirstName { get; set; }  
    public string LastName { get; set; }  
    public bool Validate(){ ... }  
}
```



Objects Are Reference Types

```
int i1;  
i1 = 42;  
  
int i2 = i1;  
i2 = 2;
```

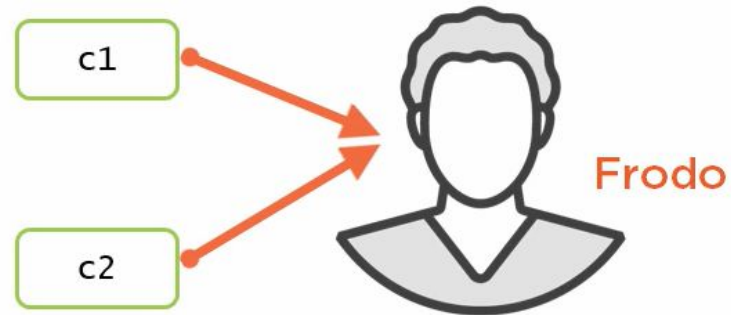
What is i1?

i1 42

i2 2

```
var c1 = new Customer();  
c1.FirstName = "Bilbo";  
  
var c2 = c1;  
c2.FirstName = "Frodo";
```

What is c1.FirstName?



ارتباط بین موجودیت‌ها



One



Many



One (and only one)



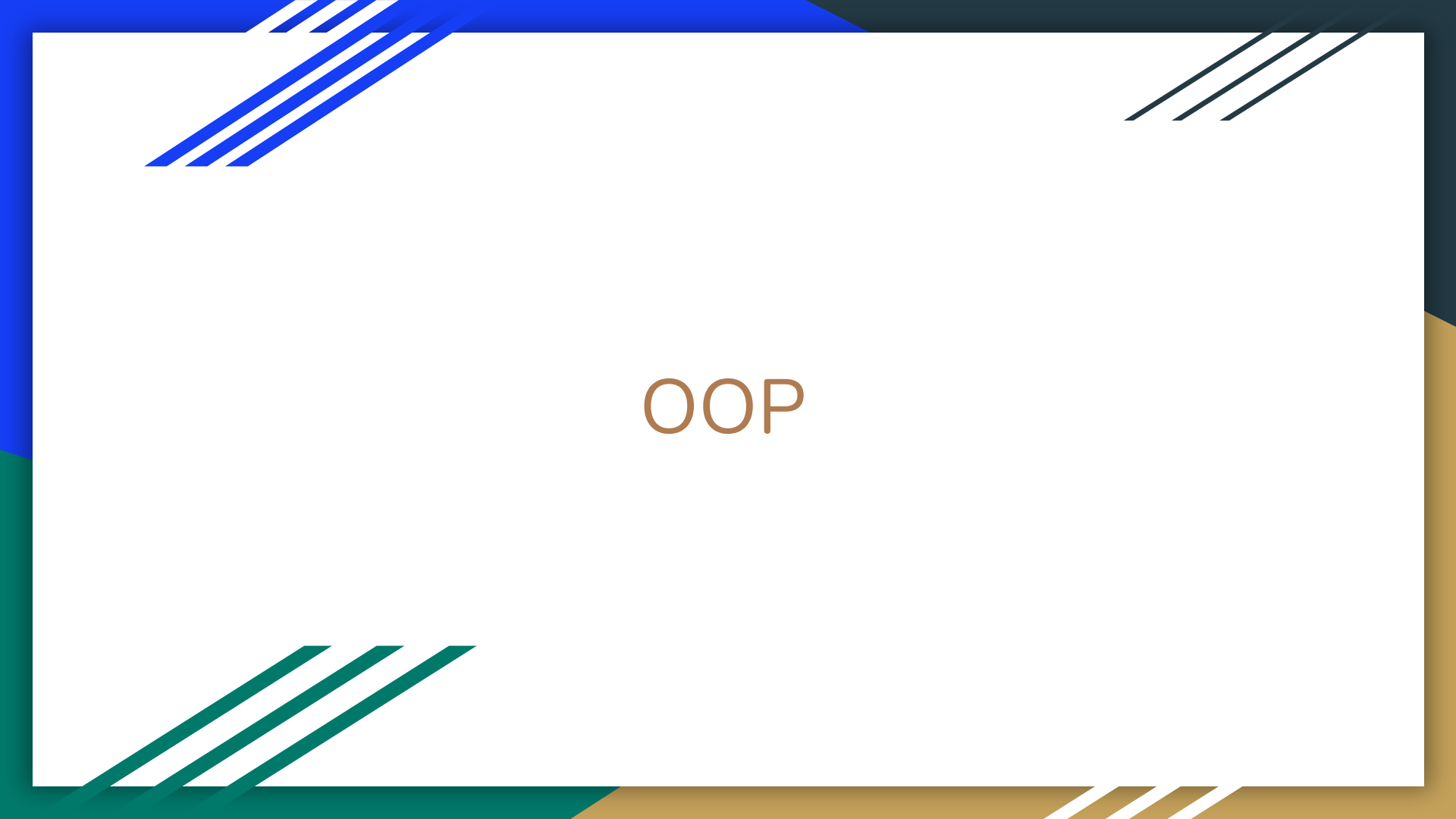
Zero or one



One or many



Zero or many



OOP

OOP

An object in the real world is a **thing**, such as a car or a person, whereas an object in programming often represents something in the real world, such as a product or bank account, but it can also be something more abstract.

Concepts of OOP:

- Encapsulation (کپسوله سازی)
- Composition
- Aggregation
- Inheritance (وراثت)
- Abstraction (انتزاع)
- Polymorphism (چند ریختی)

Abstraction-Part01

اگر من بگویم "شخص" شما می دانید منظور من چیست. من مشخص نکردم که به چه کسی فکر می کردم. من حتی شخص را توصیف نکردم، زن یا مرد، بلند یا کوتاه، شاید شخصی را به طور خاص تصور کرده باشید. خُب، ولی لازم نیست که من شخصی که شما تصور کردید رو بشناسم چون شما مفهوم شخص رو می دونید. همچنین به اندازه کافی افراد واقعی رو می شناسید و با اون ها برخورد داشتید تا مفهوم انتزاعی "شخص" رو درک کنید.

انتزاع به این معنی است که ما به جای یک مثال خاص روی ویژگیهای اساسی چیزها تمرکز می کنیم. با استفاده از انتزاع، ما به طور خودکار آنچه را بی اهمیت یا بی ربط است رو رها می کنیم. الگوی ذهنی شما از یک شخص ممکن است دارای نام، جنسیت و قد باشد، اما احتمالاً شامل عطر و مزه و تزیینات نمیشه، اینها چیزهایی هستند که اگر من توصیف یک کلوچه رو کرده بودم، منطقی بود، اما این ویژگی ها مربوط به یک شخص نیست.

انتزاع به معنای ایده یا مفهوم شخص کاملاً جدا از هر نمونه خاصی ست. این کاریه که همیشه در گفتگوها انجام میدیم، و این در قلب برنامه نویسی شی گراست چونکه این همون کاری که ما هنگام ساختن کلاس انجام میدیم. من برای هر شخصی که باید تو برنامه تعریفش کنم یک کلاس جداگانه ایجاد نمیکنم.

Abstraction-Part02

is about capturing the **core idea** of an object and **ignoring the details** or specifics.

C# has the **abstract keyword** that formalizes this concept. If a class is not explicitly abstract, then it can be described as being concrete.

Base or superclasses are often abstract; for example, the superclass Stream is abstract, and its subclasses, like FileStream and MemoryStream, are concrete.

Only concrete classes can be used to create objects; abstract classes **can only be used as the base** for other classes because they are missing some implementation.

Abstraction is a tricky balance. If you make a class more abstract, more classes will be able to inherit from it, but at the same time, there will be **less functionality to share**.

Encapsulation-Part01

ایده کپسوله سازی (Encapsulation) در مورد محتوی یک شی (Object) است. نه تنها برای نگه داشتن آنها، بلکه برای محافظت از آنها.

ما خصوصیات (attributes) یا داده های (data) یک شی را با متدهایی که بر روی آن داده ها در همان واحد یا کلاس کار می کنند، جمع (bundle) می کنیم.

در اینجا مثال ظرفی از شکلات صدق میکند. چنانچه همه خود بتوانند این دسترسی را داشته باشند چه اتفاقی خواهد افتاد؟

Encapsulation-Part02

By Reference: Encapsulation is the combination of the data and actions that are related to an object. For example, a BankAccount type might have data, such as Balance and AccountName, as well as actions, such as Deposit and Withdraw. When encapsulating, you often want to control what can access those actions and the data; for example, restricting how the internal state of an object can be accessed or modified from the outside.

Composition

By Reference : is about what an object is made of. For example, a Car is composed of different parts, such as four Wheel objects, several Seat objects, and an Engine.

Aggregation

By Reference: is about what an object is made of. For example, a Car is composed of different parts, such as four Wheel objects, several Seat objects, and an Engine.

Inheritance

By Reference: Inheritance is about reusing code by having a subclass derive from a base or superclass. All functionality in the base class is inherited by, and becomes available in, the derived class. For example, the base or super Exception class has some members that have the same implementation across all exceptions, and the sub or derived SQLException class inherits those members and has extra members only relevant to when a SQL database exception occurs, like a property for the database connection.

Polymorphism

By Reference: is about allowing a derived class to override an inherited action to provide custom behavior.

Class Members:

- Fields: are used to store data.
 - **Constant:** The data never change.
 - **Read-only:** The data can not change after the class is instantiated.
 - **Event:** The data references one or more methods that you want to execute when some-thing happens, such as clicking on a button or responding to a request from some other code.
- Methods: Are used to execute statements.
 - **Constructor:** The data references one or more methods that you want to execute when some-thing happens, such as clicking on a button or responding to a request from some other code.
 - **Property:** The statements execute when you get or set data. The data is commonly stored in a field, but could be stored externally or calculated at runtime. Properties are the preferred way to encapsulate fields unless the memory address of the field needs to be exposed.
 - **Indexer:** The statements execute when you get or set data using “array” syntax []
 - **Operator:** The statements execute when you use an operator like + and / on operands of your type.

Property

A property is simply a method (or a pair of methods) that acts and looks like a field when you want to

get or set a value, thereby simplifying the syntax.

Required Property

C# 11 introduces the required modifier. If you use it on a property or field, the compiler will ensure

that you set the property or field to a value when you instantiate it. It requires targeting .NET 7 or later

Note:.Note that all three string properties are nullable. Setting a property or field to be required does not mean that it cannot be null. It just means that it must be explicitly set to null.

Single Responsibilities

Separating responsibilities



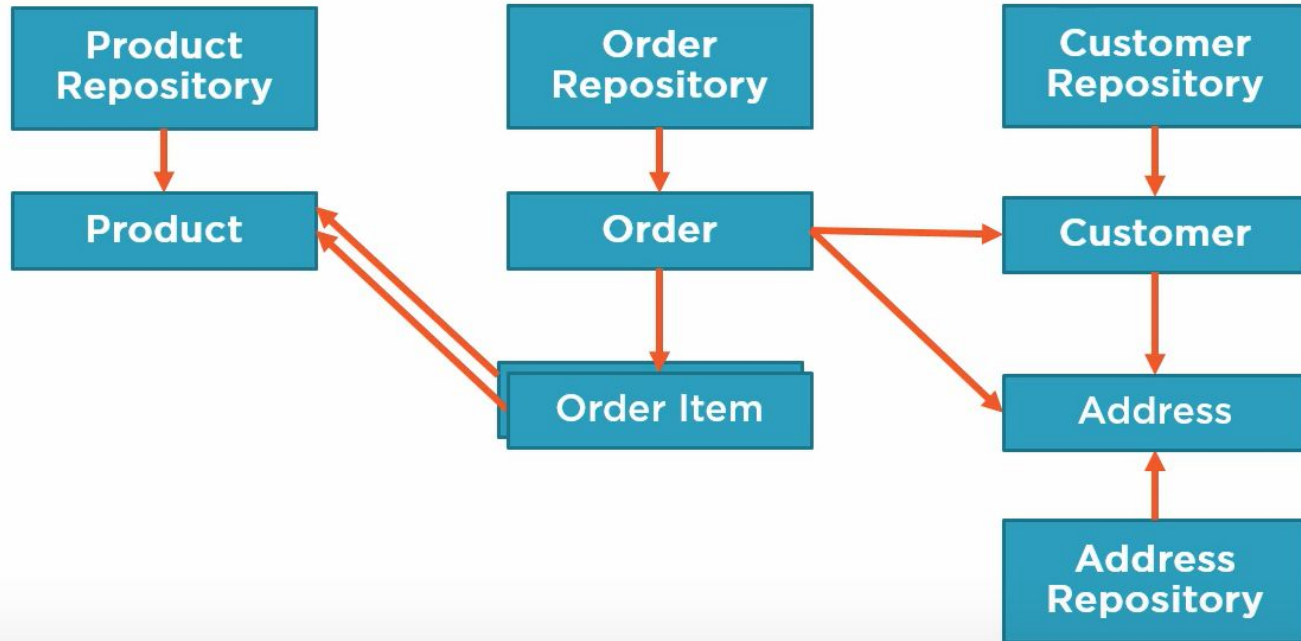
- Minimizes coupling
- Maximizes cohesion
- Simplifies maintenance
- Improves testability

Separating Responsibilities

Customer	Product	Order	Order Item
•Name •Email address •Home address •Work address •Validate()	•Product name •Description •Current price •Validate()	•Customer •Order date •Shipping addr. •Order Items •Validate()	•Product •Quantity •Purchase price •Validate()
Customer Repository	Product Repository	Order Repository	Address
•Retrieve() •Save()	•Retrieve() •Save()	•Retrieve() •Save()	•Street line 1 + 2 •City •State/Province •Postal Code •Country

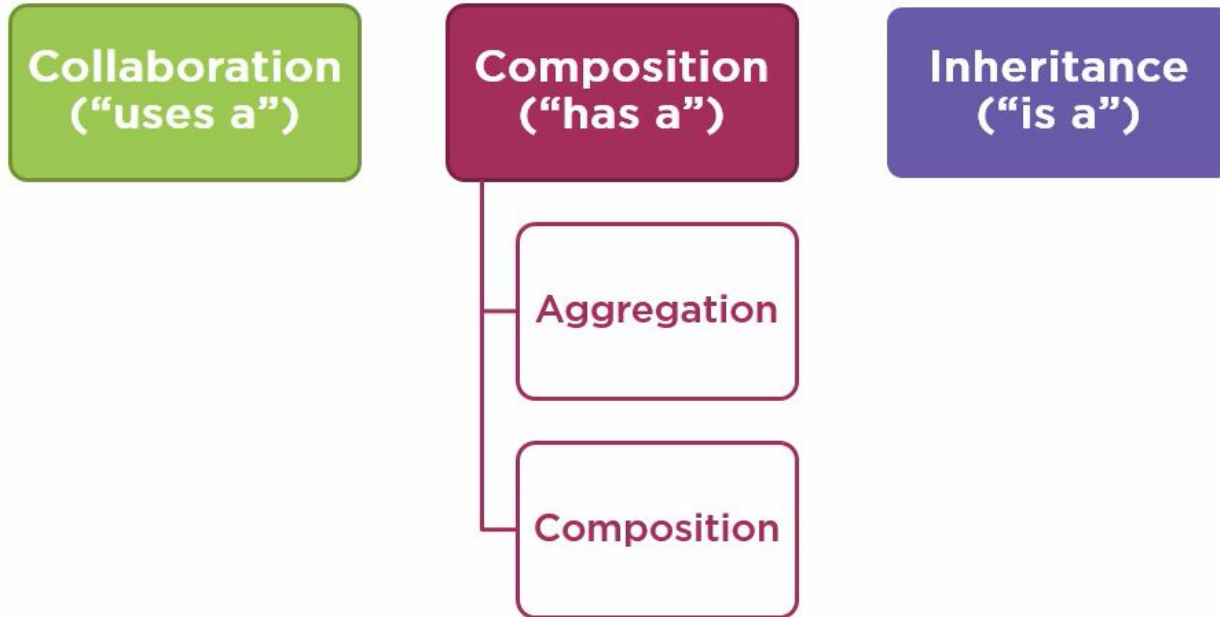
Relation in OOP

Defining Relationships



Type Of Relations

Types of Relationships



OOP Relation Example

1. A Customer (is a) type of User. -> (User is Parent if Customer) -> Inheritance
2. Order (has a) item. -> In your scenario, it could be argued that Order Items are part of an Order. -> Composition(implies a strong ownership relationship where one entity is part of another.)
3. Order item (has a) Product. -> Aggregation relationship.

C# Access Modifier

Access Modifiers are **keywords** in C# which are used to **restrict availability** of object, method, class and its members into the program or in application.

We can control the scope of the member object of a class using access specifiers which are used to provide **security** of applications.

Access modifiers in C#:

1. Public
2. Private
3. Protected
4. Internal
5. Protected internal

Public

Public is the most commonly used access specifier in C# . It can be access from **anywhere**, that means there is no restriction on accessibility. The scope of the accessibility is inside class as well as outside. The type or member can be accessed by any other code in the **same assembly** or **another assembly that references it**.

Key Points:

- within the class in which they are declared.
- within the derived classes of that class available within the same assembly.
- Outside the class within the same assembly.
- Within the derived classes of that class available outside the assembly.
- Outside the class outside the assembly.

Private

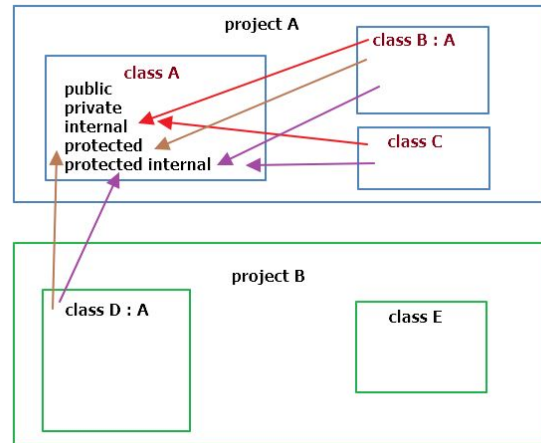
The scope of the accessibility is limited **only inside the classes or struct** in which they are declared. The private members cannot be accessed outside the class and it is the **least permissive access level**.

Key Points:

- Only Within the class in which they are declared.

Access Modifiers Note:

- Namespace will not have **access modifier**.
- **Default access modifier** for class ,struct, Interface, Enum, Delegate is **Internal**.
- **Default access modifier** for class and struct members is **private**.
- **No access modifier** can be applied to interface members and **always interface members are public**.
- **Enum members are always public** and no access modifier can be applied.



Protected

The scope of accessibility is limited within the class or struct and the **class derived from** this class.

Key points:

1. within the class in which they are declared.
2. within the derived classes of that class available within the same assembly.
3. within the derived classes of that class available outside the assembly.

Protected Internal

Protected internal is the same access levels of both protected and internal. It can access anywhere in the same assembly and in the same class also the classes inherited from the same class.

Key Points:

1. Within the class in which they are declared.
2. Within the derived classes of that class available within the same assembly.
3. Outside the class within the same assembly.
4. Within the derived classes of that class available outside the assembly.

Methods Overload

در صورتیکه چند متد هم نام اما با پارامترهای مختلف در یک کلاس قرار بگیرد به اصطلاح گفته می شود که متد ها با هم Overload شده اند.

- از جمله کاربرد های متد های Overload ارائه یک متد در حالت های مختلف می باشد تا استفاده از آن راحت تر است.
- لزومی ندارد نوع و سطح دسترسی متد ها در Overloading با یکدیگر یکسان باشند.

Method Overloading is the common way of implementing **polymorphism**.

Why do we need Method Overloading? same kind of the operation in different ways i.e. for different inputs.

Static

بنا به محیطی که استفاده میشود معنای خود را دارد:

The use of "static" is generally related to creating elements that are **shared among all instances** of a class or are **accessible without creating an instance** of the class. It's used to create utility classes, maintain global data, or define constants.

- Static Classes and Members
 - Static class
 - Static Member
- Static Constructors
- Static Methods
- Static Properties
- Static Variables

Constructor

- متدی هم نام کلاس مییاشد.
- نوع خروجی ندارد.
- وظیفه آن مقدار دهی اولیه به خصوصیات کلاس مییاشد.
- در هنگام تعریف شی از کلاس به صورت خود به خود فراخوانی میشود.
- میتوان از آن با امضاهاى متفاوت داشت.
- فراخوانی یک متد سازنده از طریق یک متد سازنده دیگر.

Different Type of constructor

1. **Default constructor**: The default constructor is a parameterless constructor that is provided **automatically** by the compiler if no other constructor is defined. It initializes all fields to their default values (0, null, false).
2. **Parameterized constructor**: A parameterized constructor is a constructor that takes one or more parameters. It is used to initialize the object with values provided by the caller.
3. **Copy constructor**: A copy constructor is a constructor that creates a new object by copying the values from an existing object. It is used to create a new object that is identical to an existing object.
4. **Static constructor**: A static constructor is a special constructor that is called only once when the class is loaded into memory. It is used to initialize static fields of the class.

Properties

- Read-Write Property
- Read-Only Property
- Static Property
- Indexer Property
 - You must need array

Syntax:

```
• //Define the Properties Accessors
• Public < type > < Property Name >
• {
•     Get
•     {
•         Return
•         <var > ;
•     }
•     Set
•     {
•         If(Is Valid(Value))
•         <var > = Value;
•     }
• }
```

Const

Immutable values which are known at **compile time** and do not change for the life of the program.

```
const double pi = 3.14159;
```

- You can not use constant for your parameters.
- You can not use constant for return value.
- Can not refer to an object.

Making a Field constant: you must write the name of class.

```
WriteLine($"{bob.Name} is a {Person.Species}");
```

this

- دسترسی به اعضای تعریف شده در کلاس.
- دسترسی به اعضای ارث برده شده در کلاس.
- تمایز قائل شدن بین اعضا کلاس و سایر متغیرها.

JSON

- JSON stands for JavaScript Object Notation
- JSON is a lightweight format for **storing and transporting data**
- JSON is often used when data is sent from a server to a web page
- JSON is "self-describing" and easy to understand

JSON Rule

- Data is in name/value pairs
- Data is separated by commas
- Curly braces hold objects
- Square brackets hold arrays

Class

Classes are **user-defined data types** that represent the **state** and **behavior** of an object. The state represents the properties, and **behavior** is the action that objects can perform.

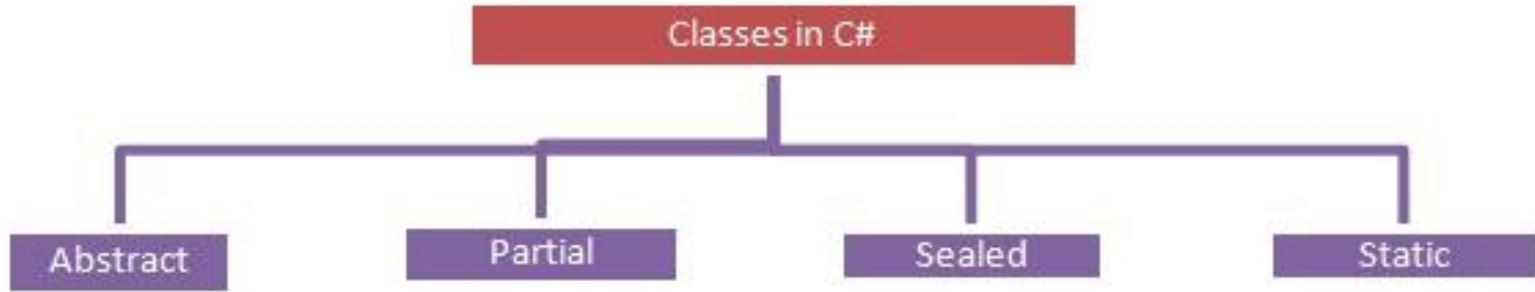
Access modifiers:

- Public
- Private
- Protected
- Internal
- Protected internal

Some Key points about classes:

- Classes are reference types that hold the object created dynamically in a heap.
- All classes have a base type of **System.Object**.
- The default access modifier of a class is **Internal**.
- The default access modifier of methods and variables is **Private**.
- Directly inside the namespaces, declarations of private classes are not allowed.

Type of class



Abstract class

An abstract class is a **special type of class** that **cannot be instantiated** and acts as a base class for other classes.

An abstract class is a class that provides a **common definition to the subclasses**, and this is the type of class whose object is not created. we use it as blueprint for other class.

Some key points of Abstract classes are:

- Abstract classes are declared using the **abstract keyword**.
- We **cannot create an object** of an abstract class.
- It must be **inherited in a subclass** if you want to use it.
- An Abstract class contains both **abstract and non-abstract methods**.
- The methods inside the **abstract class can either have an or no implementation**.
- We can inherit two abstract classes; in this case, implementation of the base class method is optional.
- An Abstract class has **only one subclass**.
- **Methods** inside the abstract class **cannot be private**.
- If there is at least one method abstract in a class, then the class must be abstract.

Partial class

It is a type of class that allows dividing their properties, methods, and events into multiple source files, and at **compile time**, these files are combined into a **single class**.

Key points:

- All the parts of the partial class must be prefixed with the partial keyword.
- If you seal a specific part of a partial class, the entire class is sealed, the same as for an abstract class.
- Inheritance cannot be applied to partial classes.
- The classes written in two class files are combined at run time.



Partial class

Key Points:

- You need to use a partial keyword in each part of the partial class.
- The name of each part of the partial class should be the same but the source file name for each part of the partial class can be different.
- All parts of a partial class should be in the same namespace.
- Each part of a partial class should be in the same assembly or DLL, in other words, you can't create a partial class in source files of a different class library project.
- Each part of a partial class has the same accessibility.
- If you inherit a class or interface on a partial class then it is inherited on all parts of a partial class.
- If a part of a partial class is sealed then the entire class will be sealed.
- If a part of the partial class is abstract then the entire class will be an abstract class.

Sealed class

Sealed classes are used to restrict the inheritance feature of object-oriented programming. Once a class is defined as a **sealed class**, this class cannot be inherited.

Key Points:

- A Sealed class is created using the sealed keyword.
- Access modifiers are not applied to a sealed class.
- To access the sealed members, we must create an object of the class.

Static Class

A static class is a class that contains only static members and cannot be **instantiated**. It is used to provide utility functions or constants that can be accessed without creating an object of the class.

Static Class	Non-Static Class
Static class is defined using static keyword.	Non-Static class is not defined by using static keyword.
In static class, you are not allowed to create objects.	In non-static class, you are allowed to create objects using new keyword.
The data members of static class can be directly accessed by its class name.	The data members of non-static class is not directly accessed by its class name.
Static class always contains static members.	Non-static class may contain both static and non-static methods.
Static class does not contain an instance constructor.	Non-static class contains an instance constructor.
Static class cannot inherit from another class.	Non-static class can be inherited from another class.

Static in Memory

چرا نیاز به یک کلاس static ممکن است وجود داشته باشد؟

همانطور که عنوان شد یک کلاس استاتیک هیچ نوع رفتاری را تعریف نمی‌کند بنابراین بهترین مکان است برای تعریف متدهای کمکی که به سایر اعضای کلاس‌های ما وابستگی نداشته، عمومی بوده، مستقل و متکی به خود هستند. عموماً متدهای کمکی در یک برنامه به صورت مکرر فراخوانی شده و نیاز است تا به سرعت در دسترس قرار داشته باشند و حداقل یک مرحله ایجاد و هله کلاس در اینجا برای راندمان بیشتر حذف گردد.

High Frequency Heap and statics

C# Tips

```
1 public static class HFP
2 {
3     public static int Number = 5;
4     public const string Name = "Value";
5
6     public static string GetValue() {
7         return Name;
8     }
9 }
```

 Mha-Karimi

Program Code

Stack Memory

Heap Memory

GC Heap

High Frequency Heap

static class

static Variables

Const Variable

Static Methods

Advantage of using getter and setter?

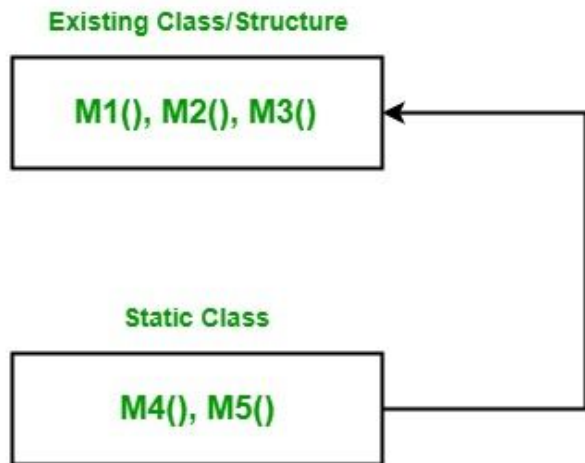
- Getters and setters provide **encapsulation** of behavior.
- Getters and setters provide a **debugging point** for when a property changes at runtime.
- Getters and setters permit different **access levels**.

Extension Method in c#

In C#, the **extension method** concept allows you to add **new** methods in the **existing class** or in the structure **without** modifying the **source code of the original type** and you do not require any kind of special permission from the original type and there is no need to re-compile the original type. It is introduced in **C# 3.0**.

حالت هایی که ممکن است ما به Extention Method نیاز داشته باشیم:

- ما دسترسی به سورس کد نداریم.
- کلاس یک sealed باشد اما باز شما بخواهید متدی به آن اضافه کنید.



Collection in C#:

- Array
- ArrayList
- List
- LinkedList
- Queue
- Stack SortedSet
- SortedDictionary
- SortedList
- Dictionary NameValueCollection
- ConcurrentBag
- ConcurrentQueue ConcurrentStack
- ConcurrentDictionary

Generic and Non-generic Collection

Generic: List, Dictionary, SortedList, Stack, Queue, HashSet.

Non-generic: ArrayList, Hashtable, SortedList, Stack, Queue

Collection by action

- List

- Dictionary

- Set

○ بر اساس index پیدا میشوند

○ بر اساس یک کلید پیدا میشوند

○ مجموعه محور هستند

Generic

Not specific to a particular data type.

Generic is a class which allows the user to define classes and methods with the placeholder. Generics were added to version 2.0 of the C# language. The basic idea behind using Generic is to allow type (Integer, String, ... etc and user-defined types) to be a parameter to methods, classes, and interfaces.

Array

An array is a **fixed-size collection** of elements of the **same type**. Arrays are useful when you need to work with a **fixed-size collection of elements**, and you know the size of the collection in advance.

Methods of Array:

CopyTo: Copies the elements of the array to another array starting at the specified index. **IndexOf:** Returns the index of the first occurrence of the specified value in the array.

Reverse: Reverses the order of the elements in the array.

Sort: Sorts the elements in the array.

Clone: Creates a new array with the same elements as the original array.

BinarySearch: Searches the array for a specified value using a binary search algorithm.

Good point:

Fixed size

continuous block of memory

provides fast element access

ArrayList

An ArrayList is a **dynamically sized** collection of objects of **any type**. ArrayLists are useful when you need to work with a collection of objects that can grow or shrink in size.

Good Point:

Resizable array

stores elements of any type

List<T>

A List<T> is a **generic dynamically sized collection** of objects of a specific type. Lists are useful when you need to work with a collection of objects that can grow or shrink in size.

Main top List<T> methods:

Add(): Adds an object to the end of the List<T>.

Contains(): Determines whether an element is in the List<T>. **Count:** Gets the number of elements actually contained in the List<T>.

Insert(): Inserts an element into the List<T> at the specified index. **Remove():** Removes the first occurrence of a specific object from the List<T>.

ToArray(): Copies the elements of the List<T> to a new array.

LinkedList<T>

A LinkedList<T> is a **generic collection** of objects of a specific type that can be efficiently inserted or removed at **any position** in the list.

Main top LinkedList<T> methods:

AddFirst(T value): Adds the specified value to the beginning of the LinkedList<T>. **AddLast(T value):** Adds the specified value to the end of the LinkedList<T>. **AddBefore(LinkedListNode<T> node, T value):** Adds the specified value before the specified node in the LinkedList<T>.

AddAfter(LinkedListNode<T> node, T value): Adds the specified value after the specified node in the LinkedList<T>.

GetEnumerator(): Returns an IEnumerator<T> for the LinkedList<T>.

Queue<T>

The **Queue<T>** class is a collection of objects arranged in a **first-in, first-out (FIFO)** order. It is useful when you want to process elements in a specific order.

Main top Queue<T> methods:

Enqueue(T): adds an element to the end of the queue.

Dequeue(): removes and returns the element at the beginning of the queue.

Peek(): returns the element at the beginning of the queue without removing it.

TrimExcess(): reduces the capacity of the queue to match the number of elements.

Stack<T>

The **Stack<T>** class is a collection of objects arranged in a **last-in, first-out (LIFO) order**. It is useful when you want to process elements in the reverse of the order in which they were added.

Main top Stack<T> methods:

Push: Adds an item to the top of the stack.

Pop: Removes and returns the item at the top of the stack.

Peek: Returns the item at the top of the stack without removing it. **CopyTo:** Copies the stack elements to an array.

GetEnumerator: Returns an enumerator for iterating through the stack elements.

HashSet<T>

The `HashSet<T>` class is a collection of **unique objects** that do not have a specific order. It is useful when you want to ensure that the elements in the collection are unique.

Main top HashSet<T> methods:

Add(T item): Adds an item to the HashSet if it does not already exist.

UnionWith(IEnumerable<T> other): Modifies the current HashSet object to contain all elements that are present in itself, the specified collection, or both.

IntersectWith(IEnumerable<T> other): Modifies the current HashSet object to contain only elements that are present in both itself and the specified collection.

ExceptWith(IEnumerable<T> other): Modifies the current HashSet object to contain only elements that are present in itself and not in the specified collection.

SortedSet<T>

A collection that stores **unique** elements in sorted order. It is implemented as a **tree-based data structure**, which provides a **fast search** and retrieval of elements.

Main top SortedSet<T> methods:

Add(T item): Adds the specified item to the sorted set.

Remove(T item): Removes the specified item from the sorted set. **Max:** Gets the maximum item in the sorted set.

Min: Gets the minimum item in the sorted set.

UnionWith(IEnumerable<T> other): Modifies the sorted set so that it contains all elements that are present in both the sorted set and the specified collection.

Mutable & Immutable

Mutable and immutable are English words that mean "**can change**" and "**cannot change**" respectively.

The meaning of these words is the same in C# programming language; that means the **mutable** types are those whose data **members can be changed after the instance is created** but Immutable types are those whose data members can not be changed after the instance is created.

Ref & Out

Sr. No.	Key	ref keyword	out keyword
1	Purpose	ref keyword is used when a called method has to update the passed parameter.	out keyword is used when a called method has to update multiple parameter passed.
2	Direction	ref keyword is used to pass data in bi-directional way.	out keyword is used to get data in uni-directional way.
3	Initialization	Before passing a variable as ref, it is required to be initialized otherwise compiler will throw error.	No need to initialize variable if out keyword is used.
4	Initialization	In called method, it is not required to initialize the parameter passed as ref.	In called method, it is required to initialize the parameter passed as out.

Understanding async and await

C# 5 introduced two C# keywords when working with the Task type that enables easy multithreading. The pair of keywords are especially useful for the following:

- Implementing multitasking for a graphical user interface (GUI).
- Improving the scalability of web applications and web services.
- Preventing blocking calls when interacting with the filesystem, databases, and remote services, all of which tend to take a long time to complete their work.

Understanding how foreach works internally

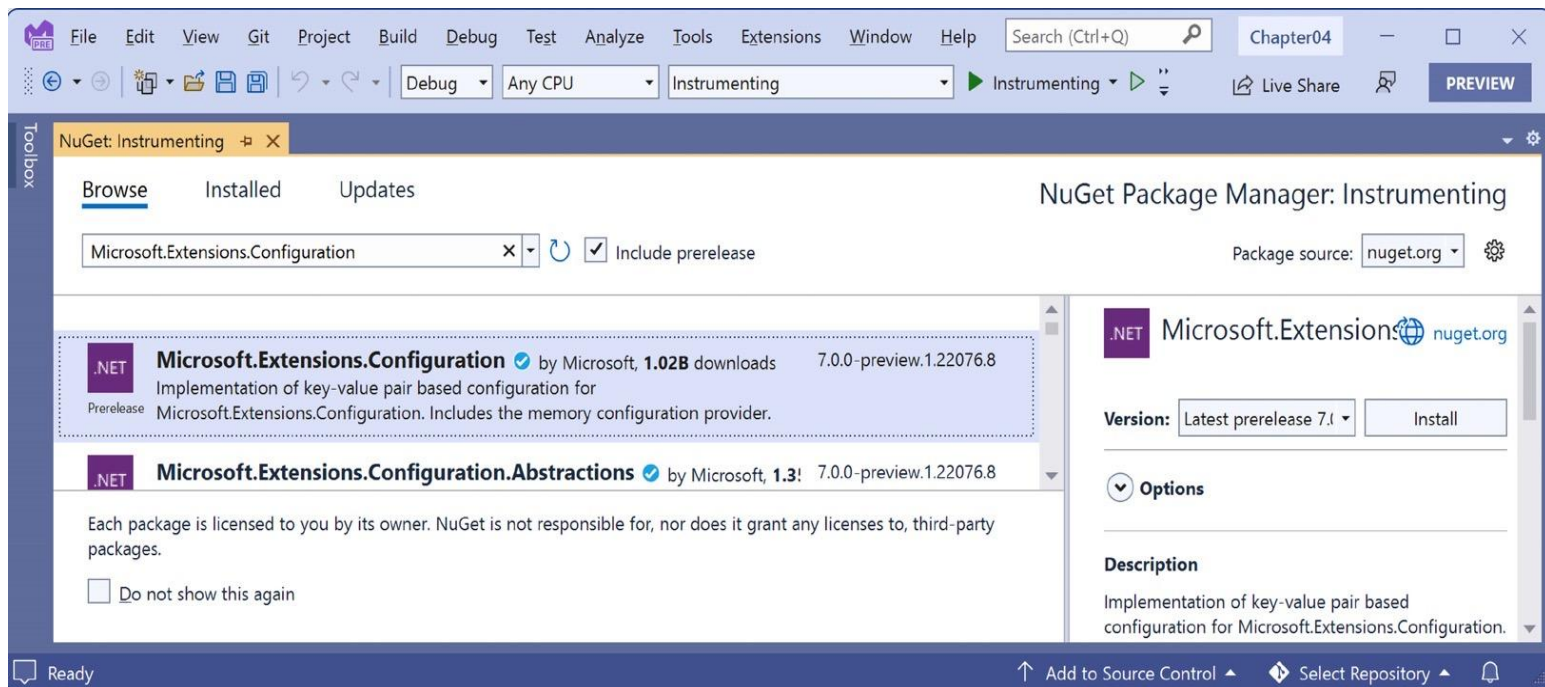
creator of any type that represents multiple items, like an array or collection, should make sure that a programmer can use the foreach statement to enumerate through the type's items.

Technically, the foreach statement will work on any type that follows these rules:

- The type must have a method named GetEnumerator that returns an object.
- The returned object must have a property named Current and a method named MoveNext.
- The MoveNext method must change the value of Current and return true if there are more items to enumerate through or return false if there are no more items.

There are interfaces named IEnumerable and IEnumerable<T> that formally define these rules, but technically the compiler does not require the type to implement these interfaces.

Adding packages to a project



Tuple in C#

C# tuple is a data structure in C#.

If you need to return multiple values from a method in C#, there are three ways to do so without using tuples.

1. Using Out parameters
2. Using a class, struct, or a record type
3. Anonymous types returned through a dynamic return type

Record

In C#, Tuples solve this problem. C# tuple is a data structure that provides an easy way to represent a single set of data. The `System.Tuple` class provides static methods to create tuple objects.

Tuples allow us to,

- Create, access, and manipulate a data set
- Return a data set from a method without using the out parameter
- Pass multiple values to a method through a single parameter

Controlling how parameters are passed into a method

By value (this is the default): Think of these as being in-only.

As an out parameter: Think of these as being out-only. out parameters cannot have a default value assigned in the parameter declaration and they cannot be left uninitialized. They must be set inside the method or the compiler will give an error.

By reference as a ref parameter: Think of these as being in-and-out. Like out parameters, ref parameters also cannot have default values, but since they can already be set outside the method, they do not need to be set inside the method.

Adding package with command

navigate to **Terminal**:

```
Dotnet add package xxxxxx
```

Types of testing

Unit: Tests the smallest unit of code, typically a method or function. Unit testing is performed on a unit of code isolated from its dependencies by mocking them if needed. Each unit should have multiple tests: some with typical inputs and expected outputs, some with extreme input values to test boundaries, and some with deliberately wrong inputs to test exception handling.

Integration Tests if the smaller units and larger components work together as a single piece of software. Sometimes involves integrating with external components that you do not have source code for.

System: Tests the whole system environment in which your software will run.

Performance: Tests the performance of your software; for example, your code must return a web page full of data to a visitor in under 20 milliseconds.

Types of testing

Load: Tests how many requests your software can handle simultaneously while maintaining required performance, for example, 10,000 concurrent visitors to a website.

User Acceptance: Tests if users can happily complete their work using your software.

Page 186 of PDF(Refrence)

Access modifiers

Access Modifier	Description
<code>private</code>	Member is accessible inside the type only. This is the default.
<code>internal</code>	Member is accessible inside the type and any type in the same assembly.
<code>protected</code>	Member is accessible inside the type and any type that inherits from the type.
<code>public</code>	Member is accessible everywhere.
<code>internal protected</code>	Member is accessible inside the type, any type in the same assembly, and any type that inherits from the type. Equivalent to a fictional access modifier named <code>internal_or_protected</code> .
<code>private protected</code>	Member is accessible inside the type and any type that inherits from the type and is in the same assembly. Equivalent to a fictional access modifier named <code>internal_and_protected</code> . This combination is only available with C# 7.2 or later.

Delegates

We use it when:

- Want to pass a function as a parameter
- Handles the callback functions or event handler
- Defines the method signature
- Declare an Event
- Declare Anonymous Type

Syntax:

```
[access modifier] delegate [return type] [delegate name] ([parameters])
```

A delegate can be declared **outside of the class or inside the class**. Practically, it should be declared out of the class.

What is Func & Action

In .NET, `Func` and `Action` types are built-in **generic delegates** that should be used for most common delegates instead of creating new custom delegates.

Func

Test class

Testing Our Code

```
public class Customer
{
    public int CustomerId { get; private set; }

    public string EmailAddress { get; set; }

    public string FirstName { get; set; }

    public string FullName
    {
        get
        {
            return LastName + "," + FirstName;
        }
    }

    private string _lastName;
    public string LastName
    {
        get
        {
            return _lastName;
        }
        set
        {
            _lastName = value;
        }
    }
}
```

Our class

```
[TestClass]
public class CustomerTest
{
    [TestMethod]
    public void FullNameTestValid()
    {
        //-- Arrange
        Customer customer = new Customer();
        customer.FirstName = "Bilbo";
        customer.LastName = "Baggins";

        string expected = "Baggins, Bilbo";

        //-- Act
        string actual = customer.FullName;

        //-- Assert
        Assert.AreEqual(expected, actual);
    }
}
```

Test for our class

Record

A **record** in C# is a **class** or **struct** that provides special syntax and behavior for working with data models.

When to use Record?

Consider using a record in place of a class or struct in the following scenarios:

- You want to define a data model that depends on **value equality**.
- You want to define a type for which objects are immutable.

Runtime vs Compile Time

Compile time is the period when the programming code (such as [C#](#), [Java](#), [C](#), [Python](#)) is converted to the [machine code](#) (i.e. binary code). **Runtime is the period of time when a program is running and generally occurs after compile time.**

